33 팀 할수있다 능



해외여행 중 악성 QR 의 위험으로부터 사용자의 안전한 접속을 돕는 스마트 어시스턴트

2376322 황규리
2317016 신예진
2271022 김태연

1. 팀 정보

과제명	해외여행 중 악성 QR 의 위험으로부터 사용자의 안전한 접속을 돕는 스마트 어시스턴트
팀 정보	33 팀(할수있다 능)
구성원	2376322 황규리(BE/AI), 2317016 신예진(FE/AI), 2271022 김태연(FE/AI)
지도 교수	심재형 교수님

2. 과제 요약

2.1. 필요성 및 배경

(1) 큐싱 보안 위협 현황

코로나 팬데믹을 거치며 전 세계적으로 QR 코드 활용이 폭발적으로 증가하였다. 글로벌 QR 코드 생성량은 2020 년 대비 꾸준히 상승해 2022 년에는 약 87% 증가하는 등 QR 은 이제 결제·주문·인증 등 일상 전반에서 보편적인 접근 수단으로 자리 잡았다. 그러나 QR 사용이 확대된 만큼 이를 악용한 큐싱(Quishing) 역시 빠르게 증가하고 있다. 큐싱은 QR 에 악성 URL 을 심어 사용자가 스캔하는 순간 피싱 사이트로 연결시키는 공격으로, 코로나 기간 동안 13 만 건 이상의 피싱 URL 이 생성되었고 2023 년 국내 QR 피싱 피해는 전년 대비 60% 증가하였다.

(2) 해외여행 맥락의 보안 취약성

팬데믹 이후 해외여행 수요는 급격히 회복되며 2024 년 기준 국민 해외관광객 수는 팬데믹 이전 수준을 넘어섰다. 여행지에서는 장소 안내, 티켓 발권 등 다양한 상황에서 QR 사용이 필수적이지만, 언어 장벽, 낯선 환경 등으로 인해 여행자는 QR 스캔 시 URL 의 진위를 스스로 판단하기 어렵다. 최근 동남아·유럽 지역에서 여행객을 노린 QR 결제 사기가 급증하고 있다는 보도 역시 이러한 취약성을 뒷받침한다. 즉, 전 세계적 QR 사용 증가와 해외여행 재개, 그리고 여행자 특유의 정보 취약성이 결합되며 해외여행자는 큐싱 공격의 주요 표적이 되고 있다.

2.2. 기존 연구와의 비교

현존하는 큐싱 방지 서비스들을 비교한 후 공통적으로 갖는 한계점을 분석해보았고, 관련 논문을 참고하여 기술적인 한계를 보완할 만한 방법에 대한 조사를 진행했다.

(1) 기존 서비스 비교 및 한계

‘KT 안심 QR’ 서비스는 KT 가 2024 년에 출시한 큐싱 예방 서비스로, ‘마이케이티’라는 앱을 통해 이용할 수 있다. 해당 서비스는 블랙리스트 기반으로 동작하며, 이미 알려진 악성 URL 에 대해 사후적으로 차단하는 방식이다. ‘KISA 보호나라’는 한국인터넷진흥원에서 운영하는 보안 서비스로, 카카오톡 ‘보호나라’ 채널에서 큐싱 메뉴를 제공한다. 해당 채널에서 사용자가 의심되는 URL 이나 QR 코드를 직접 신고하면 이를 분석하여 악성 여부를 확인해주는 구조이다.

KT 안심 QR 서비스의 경우에는 데이터베이스에 기반한 탐지 구조를 가지기 때문에, 새롭게 등장하는 악성 URL 에 대해 즉각적인 대응이 어렵다는 한계를 가진다.‘KISA 보호나라’의 경우에는 카카오톡 채널을 통해 제공되는 서비스이기 때문에 접근성이 좋지 않고, 단순히 악성 여부만 확인할

수 있으며, 구체적인 분석 결과나 판단 근거를 알 수 없다는 한계를 가진다. 또한, 두 서비스 모두 국내를 중심으로 운영되기 때문에 해외 사이트나 글로벌 환경에서 발생하는 QR 기반 위협에 대한 대응 범위가 제한적이다.

(2) 참고 논문

단순 DB 및 블랙리스트 기반 탐지 구조의 한계를 보완하기 위해, QR 코드로 연결되는 URL 의 구조적 특징을 분석하는 머신러닝 기반 탐지 기법에 관한 선행 연구를 검토했다.

첫 번째로 참고한 논문은 「URL 주요특징을 고려한 악성 URL 머신러닝 탐지모델 개발」[1]이다. 해당 연구는 다양한 머신러닝 알고리즘을 적용한 실험에서 높은 정확도와 F1-score 를 보이며, URL 의 구조적 특징을 활용한 탐지 기법의 효과성을 검증하였다. 이를 통해 URL 기반 특징 분석이 악성 URL 탐지에 있어 유의미하고 타당한 접근 방식임을 확인할 수 있었다. 또한 해당 연구에서는 문자열 기반 피처를 추출하여 악성 여부를 판단하는 방식을 제안하였으며, 본 프로젝트 이러한 방식을 참고하여 URL 의 문자열 기반 피처를 설계하였다.

「다중 머신러닝 알고리즘을 이용한 악성 URL 예측 시스템 설계 및 구현」[2]에서는 단일 머신러닝 모델의 성능 개선을 위해, 여러 모델의 예측 결과를 결합하는 다중 모델 기반 접근 방식을 제안하였다. 다중 모델 조합을 적용한 경우 단일 모델 대비 예측 정확도가 향상되는 결과를 보였으며, 모델 간 조합에 따라 성능 차이가 발생한다는 점을 통해 적절한 모델 선택과 조합이 중요함을 확인할 수 있었다. 해당 논몬에서 나타난 모델 조합별 성능 차이를 토대로 본 프로젝트에서도 모델 앙상블을 진행하였다.

2.3. 핵심 아이디어 및 솔루션

현재 있는 큐싱 방지 서비스의 경우 단순 DB 에 기반하여 판단하고 있어 새로운 악성 URL 의 경우 걸러낼 수 없다는 문제점이 존재한다. 또한 외국어로 된 웹사이트의 내용을 이해할 수 없어 해당 사이트의 목적 및 위험 여부를 알 수 없으며 해외 환경에서 사기나 정보 유출 위험에 대한 불안감을 가지게 된다는 문제점이 존재한다.

본 프로젝트는 AI 기반 URL 분석 모델을 활용하여 QR 스캔 시 발생하는 피싱 위험을 차단하고 사이트 내용 이해를 돕는 "스마트 어시스턴트"를 목적으로 한다 핵심 아이디어는 다음과 같다.

(1) 머신러닝 기반 악성 URL 탐지

URL 구조 분석과 도메인 정보를 피처로 하는 URL 분류 앙상블 모델로 정상/악성을 판별한다. 해외여행과 관련된 URL 로 모델을 파인 튜닝해 여행 시 마주할 URL 에 특화된 모델을 만든다. 사용자가 QR 코드를 스캔하면 해당 URL 의 위험도를 안전·주의·위험의 3 단계로 직관적으로 제공하며, XAI(설명 가능한 AI)를 적용해 사용자 수준에 맞는 이해 가능한 판단 근거를 함께 제시한다. 또한 리다이렉션 로그를 분석하여 중간 경유지부터 최종 목적지까지의 위험 요소를 추적함으로써, 보다 신뢰도 높은 보안 정보를 제공한다.

(2) 웹 뷰 기반 보안 브라우저

---

[1] 김영준, 이재우. (2022). URL 주요특징을 고려한 악성 URL 머신러닝 탐지모델 개발. 한국정보통신학회논문지, 26(12), 1786-1793.

[2] 강홍구, 신삼신, 김대엽, and 박순태. "다중 머신러닝 알고리즘을 이용한 악성 URL 예측 시스템 설계 및 구현." 멀티미디어학회논문지 23.11 (2020): 1396-1405.

사용자가 QR 코드를 통해 웹페이지에 접속할 때, 격리된 보안 환경에서 URL 을 로드하는 보호 계층을 제공한다. 추가로 웹페이지 번역, 위험문구 안내 등을 통해 내용 이해를 돕는다.

2.4. 기대 효과 및 의의

본 프로젝트는 해외여행자의 QR 피싱 피해를 예방하고, 보안 인식 향상 및 정보 접근성을 개선하는 것을 목표로 한다. 기존 서비스 대비 다음과 같은 효과를 기대할 수 있다.

(1) QR 피싱 대응 과정 단순화 및 사전 차단 강화

기존에는 사용자가 QR 코드 스캔 후 URL 의 안전성을 직접 판단하거나, 별도의 보안 서비스를 통해 확인해야 하는 불편함이 존재한다. 본 서비스는 QR 스캔 → 위험도 분석 → 경고 제공까지의 과정을 단일 흐름으로 통합하여, 기존 대비 보안 확인 절차를 2~3 단계 이상 감소시킬 수 있다.

(2) 사용자 보안 판단 부담 감소 및 대응 유도

기존에는 URL 구조, 도메인 등을 사용자가 직접 해석해야 하기 때문에 일반 사용자가 위험 여부를 판단하기 어렵다. 본 서비스는 XAI 기반 설명과 대응 가이드를 제공하여, 사용자의 주관적 판단 의존도를 낮추고, 실제 행동(접속 중단, 주의 등)으로 이어질 가능성을 높인다.

(3) 언어 장벽으로 인한 위험 노출 감소

해외 웹페이지 이용 시 언어 장벽으로 인해 위험 정보를 제대로 인지하지 못하는 문제가 존재한다. 웹 뷰 기반 번역 기능을 통해 별도 번역 앱 사용 없이 즉시 정보 이해가 가능하도록 하여, 정보 접근 과정을 단축한다.

2.5. 주요 기능 리스트

(1) 머신러닝 기반 악성 URL 탐지악성 URL 판별: 기존의 URL 문자열 데이터 기반 분류 앙상블 모델 생성 후 해외여행 관련 URL 로 파인튜닝해 해외여행 도메인에 특화된 모델을 생성한다.

(2) 사용자 맞춤 위험 판단 근거 제시: 설명 가능한 AI(XAI)를 활용해 URL 길이, 특수문자 사용, 도메인 구조 등 모델이 중요하게 반영한 특징을 시각화하여 위험 원인을 직관적으로 설명한다.

(3) 리다이렉션 로그 추적: URL 접속 과정에서 발생하는 모든 HTTP 리다이렉션 경로를 순차적으로 추적하여 최종 도착 URL 과 중간 경유지를 함께 분석한다.

(4) 자바스크립트 실행 제어: WebView 의 javaScriptEnabled 속성을 사용하여 페이지 로드 시 JavaScript 실행을 기본적으로 차단한 상태로 로딩하고 페이지 안전성 검토 후 필요 시에만 실행을 허용한다.

(5) 페이지 번역: Google Cloud Translation API 를 활용하여 외국어로 제공되는 웹페이지 내용을 사용자가 쉽게 이해할 수 있도록 번역한다.

(6) 민감 정보 자동 감지: 입력 필드의 구조와 텍스트를 데이터베이스 기반으로 분석하여 개인정보 입력 유도 여부를 판단하고, 위험이 감지될 경우 사용자에게 경고 메시지를 제공한다.

3. 과제 설계

3.1. 요구사항 정의

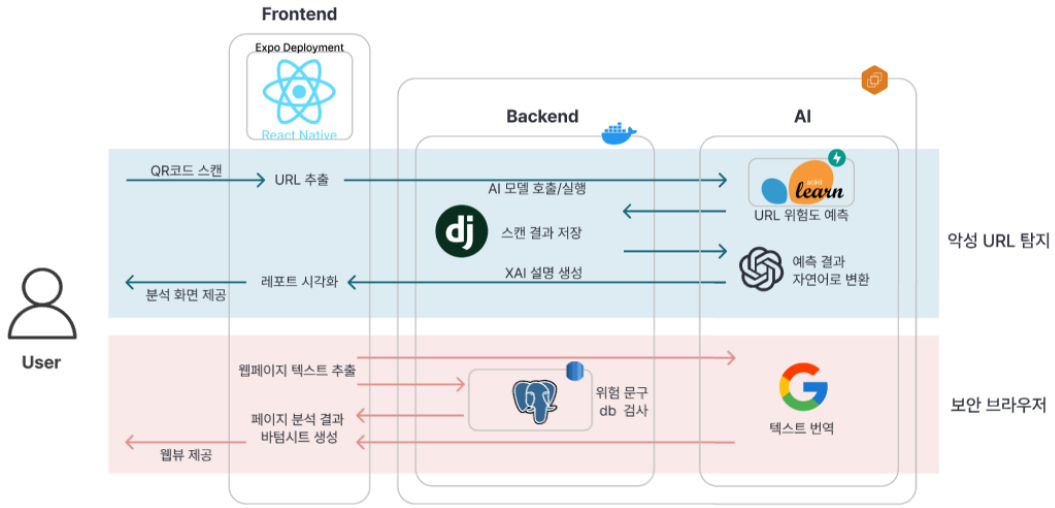
페이지 단위로 작성한 주요 기능 명세서는 아래와 같다.

페이지	기능	기능 설명
온보딩	서비스 주요 기능 및 사용 흐름 안내	악성 URL 탐지, 위험 경고, 세이프존 브라우저 기능 등 핵심 보안 기능을 시각적으로 설명함. 사용자가 별도의 보안 지식 없이도 서비스 목적과 사용 방법을 직관적으로 이해할 수 있도록 구성
여행 정보 입력	사용자 여행 정보 입력 및 개인화 보안 환경 설정	사용자가 여행 중인 국가, 도시, 여행 기간을 입력하여 서비스 이용 환경을 설정. 입력된 정보는 이후 기능에서 다음과 같이 활용됨 1) 여행 안전도 지표 제공 2) 최근 스캔 활동 기록과 연동 3) 상황 기반 보안 대응 정보(긴급 신고 및 대응 기관 정보 및 피해 발생 시 대응 가이드) 제공
QR 인식	QR 코드 인식 및 URL 추출	인식된 QR 정보는 자동으로 URL 분석 기능과 연동되어 사용자가 별도의 추가 작업 없이 즉시 보안 검사를 수행할 수 있도록 함
URL 안전성 분석 및 결과 제공	QR 코드 스캔 또는 URL 입력 시, AI 기반 모델을 통해 해당 URL 의 위험도를 분석	분석 결과를 정상/악성으로 구분하여 시각적으로 표시. 단순 결과 제공에 그치지 않고, 사용자가 판단 근거를 이해할 수 있도록 다음 정보를 함께 제공 1) 리다이렉션 로그 제공 2) AI 기반 위험 판단 근거 제공(XAI)
	사용자 선택에 따라 다음 행동을 유도 1) 정상 판별의 경우, '보안 브라우저로 열기' 버튼 클릭 시 세이프존 브라우저로 연결	

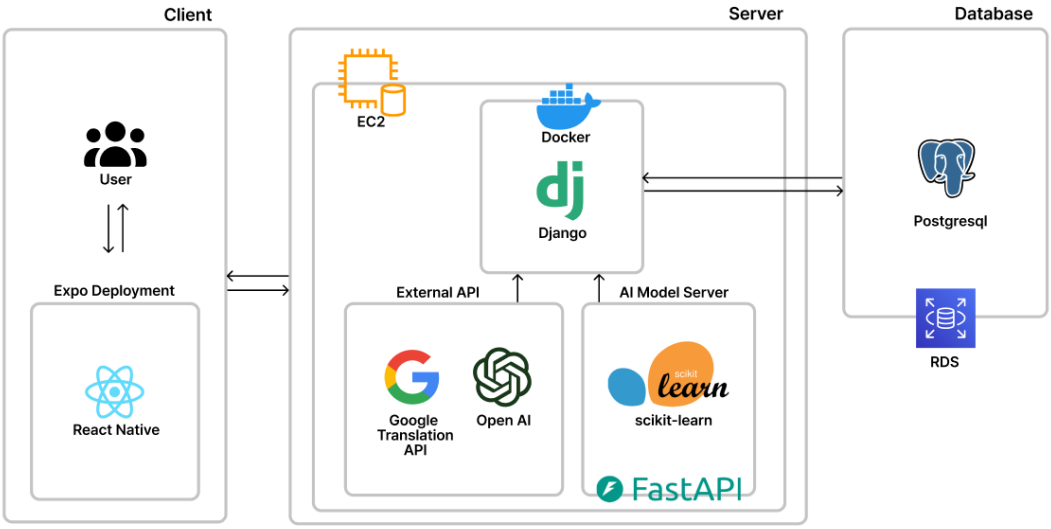
	2) 악성 판별의 경우, 경고 팝업 제공 후 기본적으로 접근 차단	
세이프존 브라우저 ON/OFF	사용자가 URL 에 접속하는 경우 자동으로 세이프존 브라우저 활성화	페이지 로드 시 다음 기능이 자동으로 적용 1) 위험 문구 탐지(로그인 유도, 결제 요청 등 피싱 의심 요소 식별) 2) 실시간 번역 기능 제공(외국어 콘텐츠 이해 지원)

3.2. 전체 시스템의 구성

3.2.1. 전체 시스템 개념도



3.2.2. 시스템 구조도



본 시스템은 React Native 기반 모바일 클라이언트, Django 기반 백엔드 서버, FastAPI 기반 AI 모델 서버, 그리고 Postgresql 기반 데이터베이스로 구성된다. 전체 시스템은 사용자 요청에 대해 URL 분석, AI 기반 위험도 판정, XAI 설명 생성 및 결과 시각화가 유기적으로 이루어질 수 있도록 설계하였다. 클라이언트 영역은 React Native 기반 모바일 애플리케이션으로 구성되며, Expo 기반 테스트 배포 환경을 활용하여 실행된다. 사용자는 모바일 환경에서 QR 코드를 스캔하고 URL 분석을 요청할 수 있으며, 분석 결과 및 리다이렉션 정보, XAI 설명 결과 등을 애플리케이션 화면을 통해 확인할 수 있다. 또한 내부 WebView 기반 보안 브라우저를 통해 분석 완료된 URL 에 안전하게 접근할 수 있도록 구성하였다.

서버 영역은 EC2 환경에서 Docker 기반 컨테이너 구조로 운영되며, Django 서버가 전체 시스템의 중심 역할을 수행한다. Django 서버는 클라이언트 요청을 수신하고 URL 데이터 검증, 리다이렉션 추적, AI 서버 연동, 결과 통합 및 응답 반환 등의 기능을 담당한다. 또한 Google Translation API 및 OpenAI API 와 연동하여 번역 기능과 사용자 친화적 위험 설명 생성 기능을 제공하도록 구성하였다.

AI 모델 서버는 FastAPI 기반으로 구현하였으며, scikit-learn 기반 머신러닝 모델을 활용하여 URL 피싱 여부를 분석하도록 설계하였다. 입력된 URL 에 대해 문자열 및 메타데이터 기반 피처를 추출하고, 학습된 앙상블 모델을 통해 악성 여부를 예측한다. 이후 SHAP 기반 XAI 분석을 수행하여 URL 의 어떤 특징이 악성 판단에 영향을 미쳤는지 설명 가능한 형태로 반환하도록 구성하였다.

데이터베이스 영역은 AWS RDS 기반 Postgresql 데이터베이스를 사용하여 구축하였다. 사용자 요청 정보, 분석 결과 데이터 및 기타 시스템 운영 데이터를 안정적으로 저장·관리할 수 있도록 설계하였다.

~~지도 교수님 피드백~~

~~앙상블 실험 진행 이후 전반적으로 앙상블 모델의 성능이 기대만큼 나타나지 않은 원인과 해석을 중심으로 지도 교수님과 면담을 진행하였다. 추가적인 모델 실험을 진행하는 것이 필요한지에 대해 질문하였고, 이에 대해 현재 시점에서는 모델 자체를 변경하거나 추가로 실험하는 것은 큰 의미가 없다는 답변을 들었다. 이미 다양한 모델과 앙상블 기법을 적용했음에도 성능 향상이 크지 않았다는 점에서, 이는 모델의 문제가 아니라 데이터의 한계에서 비롯된 것일 가능성이 높다는 의견이었다. 발전 방향에 있어, 메타데이터 기반 피처 추가와 데이터셋 재구성 중 어떤 방향을 우선적으로 개선하는 것이 효과적인지에 대해서도 질문하였고, 두 가지 모두 필요하지만 특히 피처 확장이 더 중요하다는 답변을 들었다. 또한, 일반적으로 앙상블은 단일 모델보다 성능이 나빠지기 어렵기 때문에 현재 결과에 대한 추가적인 확인이 필요하다는 피드백을 받았다.~~

~~추후 발전 방향~~

~~전반적으로 이번 실험 결과는 모델의 종류나 구조의 문제가 아니라 데이터의 질과 다양성 부족에서 비롯된 한계로 해석할 수 있으며, 향후에는 데이터 측면에서의 개선이 우선적으로 이루어져야 한다는 방향성을 도출하였다. 따라서 향후 진행 계획은 다음과 같다.~~

~~(1) 메타데이터 피처 활용~~

~~URL 의 문자열 기반 특징뿐 만 아니라 도메인 생성 시점, 인증서 정보와 같은 메타 데이터 기반 피처를 추가적으로 활용하여 학습 피처를 다양화할 필요가 있다. 이를 통해 모델이 보다 다양한 관점에서 URL 을 판단할 수 있도록 개선할 수 있을 것으로 기대된다.~~

~~(2) 학습 데이터셋 재구성~~

~~기존 데이터셋은 특정 유형의 URL 에 편중되어 있을 가능성이 있으며 이는 모델의 일반화 성능을 제한하는 원인이 될 수 있다. 따라서 다양한 출처의 데이터를 수집하고, 데이터 분포를 재구성하거나 노이즈 데이터를 정제하는 등 전처리 과정을 보다 체계적으로 수행할 필요가 있다~~

3.3. 주요 엔진 및 기능 설계

3.3.1. AI 엔진

본 프로젝트는 QR 코드 스캔을 통해 추출된 URL 이 정상 사이트인지, 피싱 사이트인지 판별하기 위해 머신러닝 기반의 악성 URL 탐지 모델을 핵심 기술로 사용한다. 블랙리스트 기반 방식으로는 신종 공격과 변조된 URL 을 효과적으로 잡아내기 어렵기 때문에, URL 자체의 형태적 패턴을 학습하는 ML 접근이 필수적이라고 판단하였다.

3.3.1.1. 머신러닝

① 모델 선정 및 구성

초기에는 URL 문자열의 다양한 구조적 특징을 비선형적으로 해석할 수 있는 RandomForest 를 기반 모델로 채택하였다. RandomForest 는 여러 개의 결정트리를 조합하는 앙상블 모델로, 피처 간 상대적 중요도를 파악하기 용이하여 모델의 판단 근거를 설명하고자 하는 본 프로젝트의 XAI 방향성과도 잘 부합한다.

그러나 단일 모델의 성능 한계를 극복하고 탐지 정확도를 높이기 위해 추가적인 모델 비교 실험을 진행하였다. 그래디언트 부스팅 계열의 LightGBM 과 XGBoost 를 도입하여 각 모델의 성능을 평가한 결과, 모델별로 탐지에 강한 패턴이 상이함을 확인하였다. 이에 세 모델을 앙상블하는 방식을 최종 채택하여 개별 모델의 약점을 상호 보완하고 전반적인 탐지 성능을 향상시켰다.

② 피처 선정

모델 학습에는 총 24 개의 문자열 피처가 사용되었다. 문자열 피처는 각 URL 이 가지는 구조적 특성을 정량화해 악성 여부를 학습하는 데 활용된다. 악성 URL 은 대부분 길이가 길다는 특성과, 복잡한 서브도메인·IP 주소 기반으로 위장하기 위해 특수 문자가 과도하게 사용될 수 있다는 특성 등을 고려하여 구성된 피처들이다.

③ 사용 라이브러리

전처리가 완료된 데이터는 scikit-learn 의 train_test_split 을 이용해 학습용과 테스트용으로 분리하며, shuffle=True 를 설정해 데이터 편향을 최소화한다. 모델 학습에는 RandomForestClassifier 를 사용해 피처 기반 악성 URL 탐지 모델을 구축하고,

accuracy_score, precision_score, recall_score, f1_score 등의 평가 지표를 활용해 baseline 모델의 성능을 파악한다.

3.3.1.2. 도메인 파인튜닝

본 프로젝트의 목적은 해외여행자가 QR 코드를 통해 접속하는 웹페이지 중 악성 사이트의 위험을 사전에 차단하는 것이다. baseline 모델은 전반적인 정상·악성 URL 패턴을 높은 정확도로 학습하고 있으나, 여행 도메인만의 고유한 패턴은 충분히 반영되지 않았다는 한계가 존재한다. 따라서 baseline 모델만으로는 여행 환경에서 발생하는 공격 시나리오 전체를 포괄하기에 한계가 있으며, 이를 보완하기 위해 해외여행 특화 데이터를 추가적으로 학습시키는 파인튜닝 전략이 필요하다고 판단하였다.
파인튜닝은 baseline 모델이 이미 학습한 일반 URL 패턴 위에, 해외여행과 밀접한 URL 패턴을 추가 학습시켜 특정 도메인에서의 탐지 정밀도 향상(domain specialization)을 목표로 한다. baseline 모델과 동일한 데이터 전처리 및 머신러닝 라이브러리를 사용한다.

① 데이터 추가 확보

1) baseline 모델은 Cisco Popularity List(정상)와 Phishtank(악성)를 기반으로 구성
2) fine-tuning 단계에서 Kaggle Malicious URLs Dataset 과 Cisco Popularity List 로부터 travel 관련 URL 을 선별하여 정상/악성 각각 400 개씩 확보
3) travel keyword 기반 필터링 코드 적용("flight", "booking", "visa", "tour", "reservation" 등)

② 전처리 및 feature extraction 규칙 통일

baseline 모델과 fine-tuning 모델 간의 구조적 일관성을 유지하기 위해, 동일한 URL 피처 추출 규칙을 적용한다. 이로써 파인튜닝 이후에도 모델의 구조적 안정성을 확보한다.

3.3.2. 프론트엔드 엔진 설계

프론트엔드 엔진은 사용자와 시스템 간의 직접적인 상호작용을 담당하도록 설계하였다. QR 코드 촬영을 통해 사용자의 URL 분석 요청을 입력받고, 백엔드 서버로부터 수신한 분석 결과를 시각적으로 표시하는 역할을 수행한다. 분석 결과 표시 과정에서 피싱 판정 결과 및 XAI 기반 위협 요소 설명을 사용자가 직관적으로 이해할 수 있는 형태로 가공하여 제공하며, WebView 를 통해 실제 URL 접근 전 페이지 미리보기 기능도 지원한다. 프론트엔드 엔진은 크게 QR 스캔 및 URL 추출 모듈, URL 위험도 분석 요청 모듈, URL 분석 결과 시각화 모듈, 보안 브라우저 모듈로 구성된다.

3.3.2.1. QR 코드 스캔 및 URL 추출 모듈

모바일 기기의 카메라를 활용하여 QR 코드를 스캔하고, 내부에 포함된 URL 정보를 추출하여 서버 분석 단계로 전달한다.

주요 기능

- QR 코드 인식 및 URL 추출
- QR 스캔 결과 전달

구현 내용 및 방법

본 모듈에서는 모바일 카메라 기반 QR 인식 기능을 활용하여 QR 코드 데이터를 추출하도록 구성하였다. 추출된 URL 은 백엔드 서버로 전달되어 위험도 분석이 수행되도록 설계하였다. 실시간 카메라 화면을 렌더링하고 QR 코드를 인식하도록 구성하였고, 사용자가 QR 코드를 스캔하면 스캔된 URL 데이터를 URL 분석 결과 시각화 모듈로 전달하여 이후 악성 URL 검사 및 결과 출력 과정이 수행되도록 구현하였다. 애플리케이션 실행 시 사용자에게 카메라 접근 권한을 요청하고, 권한 허용 여부에 따라 카메라 기능이 동작하도록 구현하였다.

3.3.2.2. URL 위험도 분석 요청 기능

클라이언트에서 추출한 URL 정보를 서버로 전달하고, AI 모델 기반 위험도 분석 결과를 수신한다.

주요 기능

- URL 유효성 검증
- 서버 기반 위험도 분석 요청
- 분석 결과 수신 및 상태 관리

구현 내용 및 방법

본 모듈에서는 초기 화면 렌더링 시 URL 검증 및 서버 분석 요청이 자동으로 수행되도록 구현하였다. 먼저 스캔된 URL 문자열의 형식을 자동으로 보정한 뒤 URL 유효성을 검사하여, 올바르지 않은 URL 인 경우 서버로 분석 요청을 전송하지 않도록 설계하였다. 이를 통해 불필요한 서버 요청을 최소화하였다.
유효한 URL 에 대해서는 저장된 JWT 토큰을 활용하여 백엔드 서버로 URL 위험도 분석 요청을 수행하도록 구현하였다. 서버로부터 수신한 분석 결과 데이터는 URL 분석 결과 시각화 모듈에 전달되며, 통신 과정에서 오류가 발생하거나 응답 데이터를 수신하지 못한 경우에는 오류를 사용자에게 안내하도록 구현하였다.

3.3.2.3 URL 분석 결과 시각화 모듈

결과 시각화 모듈은 AI 분석 결과와 위험 설명 정보를 사용자에게 제공하는 역할을 수행한다.

주요 기능

- 위험도 결과 표시
- XAI 설명 시각화
- 사용자 알림 제공

구현 내용 및 방법

정상적으로 분석 데이터를 수신한 경우, 화면에는 URL 의 악성 여부, 리다이렉션 로그, URL 복사 기능, AI 기반 판단 근거 정보를 제공하도록 구현하였다. 먼저 URL 의 분석 결과는 색상을 통해 정상 및 위험 상태를 직관적으로 구분할 수 있도록 구성하여 사용자가 URL 의 위험 여부를 한눈에 확인할 수 있도록 하였다.

또한 URL 결과 버튼을 클릭하면 리다이렉션 로그가 출력되도록 구현하였다. 이를 통해 사용자는 리다이렉션 횟수, 이동 과정에서 거쳐간 URL 목록, 각 단계의 HTTP 상태 코드 등을 확인할 수 있다. URL 복사 기능은 별도의 복사 버튼 UI 를 통해 제공하였으며, 버튼 클릭 시 QR 코드에서 추출된 URL 이 클립보드에 저장되도록 구현하였다. 마지막으로 화면 하단에는 서버로부터 전달받은 XAI 기반 판단 근거를 렌더링하여, AI 모델이 해당 URL 을 악성 또는 정상으로 분류한 이유를 사용자에게 시각적으로 제공하도록 구성하였다.

3.3.2.4 보안 브라우저 모듈

보안 브라우저 모듈은 분석이 완료된 URL 을 안전하게 열람할 수 있는 환경을 제공하는 역할을 수행한다.

주요 기능

- JavaScript 제어(활성화 / 비활성화)
- 페이지 탐색 및 새로고침
- 번역 및 위험 문구 탐지 인터페이스 제공

구현 내용 및 방법

react-native-webview 를 기반으로 보안 브라우저를 구현하였다. 보안 브라우저는 헤더 영역, URL 표시 영역, WebView 영역, 바텀 시트 영역의 네 가지 구성 요소로 이루어진다. 헤더 영역에는 뒤로가기, 새로고침, 닫기 버튼을 배치하였다. 뒤로가기의 경우 WebView 내부 탐색 이력이 존재하면 이전 페이지로 이동하고, 이력이 없으면 앱 내 이전 화면으로 복귀하도록 onNavigationStateChange 를 활용하여 처리하였다. URL 표시 영역에는 현재 접속 중인 URL 과 JavaScript 활성화 버튼을 배치하였다. 보안 측면에서 WebView 는 기본적으로 JavaScript 를 비활성화한 상태(javaScriptEnabled: false)로 페이지를 로드하여 동적 악성 스크립트의 실행을 원천 차단한다. 해당 상태에서 페이지를 충분히 검토한 후에 사용자가 필요 시 JavaScript 활성화 버튼을 통해 직접 활성화할 수 있도록 구성하였다. JavaScript 활성화 여부가 변경될 때 WebView 컴포넌트의 key 속성을 변경하여 컴포넌트를 재마운트함으로써 변경된 보안 설정이 즉시 적용되도록 하였다. 하단에는 커스텀 바텀 시트를 배치하여 번역 및 위험 문구 탐지 기능을 위한 인터페이스를 제공한다. 바텀 시트 내부는 번역 탭과 위험 문구 탭으로 구성되며, 위험 문구 탐지 기능과의 연동을 위한 인터페이스로 활용될 예정이다.

3.3.3. 백엔드 엔진 설계

백엔드 엔진은 프론트엔드와 AI 엔진 사이의 데이터 처리 및 통신을 담당하도록 설계하였다. QR 스캔을 통한 사용자의 URL 분석 요청을 수신하고, AI 서버와 연동하여 분석 결과를 반환하는 역할을 수행한다. 분석 결과 반환 과정에서 URL 리다이렉션 추적 및 XAI 분석 기능을 처리하도록 구성하였다. 백엔드 엔진은 크게 API 처리 모듈, AI 연동 모듈, URL 분석 모듈로 구성된다.

3.3.3.1. API 처리 모듈

API 처리 모듈은 클라이언트와 서버 간 요청 및 응답을 관리하는 역할을 수행한다.

주요 기능

- 분석 요청 처리 및 응답 데이터 반환
- 요청 데이터 관리

구현 내용 및 방법

프론트엔드로부터 전달된 URL 데이터를 수신하고, 분석 결과를 JSON 형태로 반환할 수 있도록 REST API 구조로 설계하였다. 클라이언트 요청이 들어오면 입력값의 유효성을 검증한 뒤 AI 연동 모듈과 URL 분석 모듈로 작업을 위임하고, 각 모듈의 처리 결과를 통합하여 단일 응답으로 반환한다. 이를 통해 프론트엔드는 분석 로직의 내부 구조와 무관하게 일관된 인터페이스로 결과를 수신할 수 있다.

3.3.3.2. AI 연동 모듈

AI 연동 모듈은 백엔드 서버와 AI 분석 서버 간 데이터 송수신을 담당한다.

주요 기능

- AI 서버 요청 전송분석
- 결과 수신 및 결과 데이터 전달

구현 내용 및 방법

URL 분석 요청 데이터를 AI 서버로 전달하고, 머신러닝 모델의 예측 결과 및 SHAP 기반 XAI 데이터를 수신하여 후처리하도록 구성하였다. AI 서버가 반환하는 원시 데이터에는 피싱 판정 결과, feature 별 SHAP 기여도 수치가 포함되며, 이를 자연어 설명 생성 단계로 전달하기 위한 데이터 정제 과정을 수행한다. 백엔드가 AI 서버와의 통신을 전담함으로써 프론트엔드는 모델 구조나 AI 서버 주소에 의존하지 않아도 되며, 추후 AI 모델 교체 시에도 프론트엔드 수정 없이 백엔드만 변경하면 된다는 유지보수 측면의 이점이 있다.

3.3.3.3. URL 분석 모듈

URL 분석 모듈은 URL 의 이동 경로 및 추가 정보를 분석하는 역할을 수행한다.

주요 기능

- Redirect 추적
- 최종 URL 분석
- URL 이동 경로 분석

구현 내용 및 방법

사용자가 입력한 URL 이 실제로 어떤 경로를 거쳐 최종 목적지에 도달하는지를 추적하도록 설계하였다. 피싱 사이트는 정상 URL 로 위장한 뒤 다단계 리다이렉션을 통해 악성 페이지로 유도하는 방식을 자주 사용하기 때문에, 입력 URL 만으로는 위험 여부를 판단하기 어렵다. 이에 본 모듈에서는 리다이렉션 발생 횟수, 경유 URL 목록, 최종 도달 URL 을 수집하여 AI 연동 모듈 및 자연어 설명 생성 단계에 함께 전달한다. 이를 통해 표면적으로는 정상처럼 보이는 URL 도 실질적인 이동 경로 기반으로 위협을 탐지할 수 있다.

3.4. 주요 기능의 구현

3.4.1. AI 모델 설계

QR 코드 스캔을 통해 추출된 URL 이 정상 사이트인지, 피싱 사이트인지 판별하기 위해 머신러닝 기반의 악성 URL 탐지 모델을 핵심 기술로 사용한다.

A. 데이터 수집

1) baseline: 글로벌 웹사이트 순위 정보를 제공하는 'Cisco Popularity List'에서 정상 URL 약 2 천 개, 커뮤니티 기반의 피싱 URL 데이터베이스 플랫폼 'Phishtank'에서 악성 URL 약 2 천 개를 확보하였다. Phishtank 는 사용자의 제보에 의해 매일 리스트가 업데이트되기 때문에 신종 악성 URL 학습에 적합하다고 판단하였으며, 아직 유효한 URL 만 필터링하는 것이 가능하기 때문에 현재는 사라진 도메인으로 인해 모델이 불필요하게 노이즈를 학습하는 문제를 최소화하도록 했다. 또한, 길이가 200 자를 넘어가는 URL 은 이상치 데이터로 판단하여 삭제하였다. URL 주소만 주어지면 나머지 피처는 직접 추출해야 하는 상태였기 때문에 결측치는 없었다.

2) fine-tuning: 파인튜닝 단계에서는 baseline 모델이 학습하지 못한 해외여행 도메인 특화 URL 패턴을 추가로 반영하는 것을 목표로 하였다. 이를 위해 정상 URL 은 기존과 동일한 기준을 유지하면서도 해외여행 관련 키워드가 포함된 URL 을 확보하기 위해 'Cisco Popularity List'에서 400 개를 선별하였다. 악성 URL 은 Kaggle 의 Malicious URLs Dataset 을 활용하여 400 개를 확보하였다. Kaggle 데이터셋은 다양한 공격 유형(피싱, 스팸, 악성 리디렉션 등)이 혼합되어 있어 해외여행 시에 마주할 수 있는 여러 악성 패턴을 포괄할 수 있다는 장점이 있다. baseline 학습에 사용된 Phishtank 기반 악성 URL 이 주로 피싱 중심인 것과 달리, Kaggle 데이터는 문자열 난독화, 우회 리디렉션, 도메인 구조 변형 등 다양한 유형을

포함하고 있어 baseline 모델이 커버하지 못한 패턴을 추가 학습하는 데 적합하다고 판단하였다. 해당 데이터셋에서 해외여행 관련 URL 을 선별하기 위해서 키워드 리스트("travel", "trip", "tour", "flight", "airport", "airline", "booking", "hotel", "visa", "passport" 등)를 기반으로 URL 문자열을 필터링하는 코드를 적용하였다. 이를 통해 'Cisco Popularity List'와 Kaggle 데이터셋에서 관련 표현이 포함된 URL 을 식별하였으며, 단축 URL("bit.ly", "t.co", "goo.gl", "tinyURL.com" 등)에 대해서 별도의 도메인 리스트를 활용하여 정보가 왜곡되지 않도록 처리하였다.

B. 피처 엔지니어링

본 프로젝트에서는 URL 문자열로부터 총 24 개의 피처를 추출하였으며, 이를 ①길이 기반 피처, ②구조 기반 피처, ③문자 특수문자 기반 피처로 분류하였다.

① 길이(length) 기반 피처

- len_URL: 전체 URL 길이
- len_hostname: 호스트명 길이
- len_TLD: 최상위 도메인(TLD) 길이
- len_path: path 길이
- len_first_dir: 첫 번째 디렉토리 이름 길이

② 구조(structure) 기반 피처

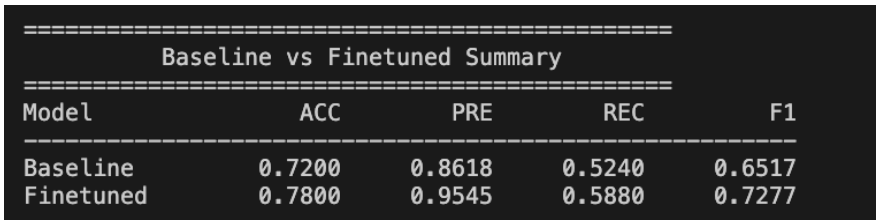
- URL_depth: 경로(path)의 디렉토리 깊이
- num_subdomains: 서브도메인 개수
- is_ip: IP 주소 사용 여부
- is_short_URL: 단축 URL 여부

③ 문자(character) 및 특수문자 기반 피처

- URL_depth: 경로(path)의 디렉토리 깊이
- num_digits: 숫자 수
- num_letters: 알파벳 수
- num_http, num_https, num_www: 특정 문자열 등장 횟수

- num_@, num_?, num_&, num_%, num_#, num_., num_=, num__, num_-: 특수문자 등장 횟수
- num_hostname_-: 호스트명 내 '-' 등장 횟수

3.4.1.1. 1, 2 차 RF 단일 모델 실험



```
============================================
        Baseline vs Finetuned Summary
============================================
Model              ACC        PRE        REC         F1
--------------------------------------------------------
Baseline         0.7200     0.8618     0.5240     0.6517
Finetuned        0.7800     0.9545     0.5880     0.7277
```

제한된 데이터 기반의 1 차 실험 이후, 데이터셋을 확장하여 2 차 실험을 수행하였다. 2 차 실험의 결과 fine-tuned 모델이 모든 주요 지표에서 baseline 보다 더 나은 성능을 보였다는 점에서 의미 있었으나, 전반적인 성능이 1 차보다 떨어진 점이 발견되었다. 이러한 현상의 원인을 분석해본 결과, 1 차 학습 데이터는 비교적 단순 구조에 편중되어 있어 트리 기반 모델이 정상·악성 패턴을 빠르게 학습하기 쉬웠다. 그러나 2 차 실험의 데이터 확장 과정에서 정상 URL 의 다양성이 증가하여 분류 난이도가 상승하였다. 2 차 실험에서 정상 URL 8,000 개를 확장하면서 다음과 같은 변화로 인해 정확도가 감소하였음을 확인할 수 있었다.

- 쿼리 파라미터 구조 증가
- path depth 다양화
- 다국적 도메인 포함
- 긴 URL + 복잡한 구조 증가

그러나 오히려 이러한 2 차 실험의 결과가 1 차 모델의 높은 성능이 실제 성능이 아니라 데이터 편향과 구조 단순성에서 비롯된 overfitting 이었다는 것의 반증이라고 판단하였다.

3.4.1.2. RF 단일 모델의 한계

RF 모델은 구현이 비교적 간단하고 안정적인 성능을 보이기 때문에, 초기 실험 단계에서 모델의 전반적인 성능 경향을 파악하고 피처의 유효성을 검증하는 데 적절한 선택이었다. 그러나 학습 데이터의 양을 늘려 진행한 2 차 실험의 결과, 해당 Accuracy 수준은 실제 서비스에 적용하기에는 여전히 부족한 것으로 판단되었다.

3.4.1.3. 앙상블 적용 배경

이에 대해 단일 모델 기반 접근만으로는 데이터 규모가 확장될 경우 성능 향상에 한계가 존재할 수 있다는 점이 지적되었고, 모델의 기본적인 capacity 를 높이기 위해 보다 다양한 모델을 실험하거나, 서로 다른 특성을 가진 모델을 결합하는 앙상블 기법을 적용하는 방안을 고려하게 되었다. 또한, 향후 학습 데이터 규모 확장, 해외여행 특화 데이터 추가, 메타데이터 기반 피처 확장이 예정되어 있다는 점을 고려할 때, 단일 모델보다 다양한 패턴을 포착할 수 있는 구조가 필요하다고 판단하였다

3.4.1.4. 앙상블 진행 과정

앙상블 모델 구성을 위해 서로 다른 학습 방식과 특성을 갖는 세 가지 후보 모델(XGBoost, LightGBM, SVM)을 선정하였다. 이를 통해 각 모델의 장점을 결합하고 성능 향상을 도모하고자 하였다.

앙상블 후보 모델에 대해서는 Random Forest 모델에 수행한 2 차 실험과 동일한 학습 데이터셋 및 테스트 데이터셋을 사용하여 개별 모델의 성능을 평가하였다. 사용된 데이터셋은 다음과 같다.

(1) Baseline 모델 학습
- 정상 URL 데이터 8,000 개
- 악성 URL 데이터 8,000 개

(2) Fine-tuned 모델 학습
- 정상 URL 데이터 400 개
- 악성 URL 데이터 400 개

(3) 테스트 데이터셋
- 정상 URL 데이터 250 개
- 악성 URL 데이터 250 개
-

선행 연구를 통해 다수 모델을 조합하는 앙상블 방식의 경우 전체 예측 소요 시간은 모델 중 가장 시간이 오래 걸리는 모델에 종속된다는 점을 확인하였다. 이에 따라, Randomforest, XGBoost, LightGBM 모델과 비교하여 성능 차이가 크지 않으면서 계산 비용이 높은 SVM 모델은 앙상블에서 제외하였다.

앙상블 방식으로는 Voting 과 Stacking 두 가지를 고려하였다. Voting 방식은 여러 모델의 예측 결과를 결합하여 최종 예측을 도출하는 기본적인 앙상블 기법으로, 구현이 간단하면서도 안정적인 성능 향상을 기대할 수 있다. Voting 방식 중 Soft Voting 과 Weighted Voting 을 비교한 결과, 세 단일 모델의 성능이 유사한 수준이었기 때문에 특정 모델에 가중치를 부여하는 것은 큰 의미가 없다고 판단하였다. 이에 따라 각 모델의 예측 확률을 평균하여 최종 결과를 도출하는 Soft Voting 방식을 적용하였다. 한편, Stacking 방식은 각 모델의 예측 결과를 메타 모델이 다시 학습하는 구조로, 모델 간 상호 보완적인 패턴을 반영할 수 있다는 점에서 적용하였다.

A. Voting 앙상블

```
================================================
        Ensemble Test Performance Summary
================================================
Model              ACC        PRE        REC         F1
------------------------------------------------------------
Voting          0.7173     0.9800     0.5880     0.7350
```

Voting 기반 앙상블 모델의 테스트 결과, Accuracy 는 0.717 로 나타났다. Precision 은 0.98 로 매우 높았으나, Recall 은 0.588 로 상대적으로 낮은 값을 보였다. 이는 모델이 악성 URL 을 보수적으로 판단하면서 일부를 정상으로 분류하는 경향이 있음을 의미한다. 결과적으로 F1-score 또한 0.753 수준에 머물렀다. 이러한 결과는 단일 모델 대비 성능 향상이 나타나지 않았음을 보여주며, 단순한 모델 결합만으로는 성능 개선이 이루어지기 어려울 수 있음을 시사한다.

B. Stacking 앙상블

```
================================================
          Model Performance Summary
================================================
Model              ACC        PRE        REC         F1
------------------------------------------------------------
Randomforest    0.7840     0.9551     0.5960     0.7340
XGBoost         0.7880     0.9675     0.5960     0.7376
lightGBM        0.7860     0.9735     0.5880     0.7332
Stacking        0.7860     0.8907     0.6520     0.7529
```

Stacking 기반 앙상블 모델의 테스트 결과, Accuracy 는 0.786 으로 나타났다. Precision 은 0.8907 로 비교적 높은 값을 유지하여, 악성 URL 로 판단한 경우 실제로 악성일 확률이 높은 것으로 나타났다. 반면 Recall 은 0.652 로 상대적으로 낮은 수치를 보여, 일부 악성 URL 을 놓치는 경향이 있음을 확인할 수 있다. 이에 따라 F1-score 는 0.7529 수준으로, Precision 대비 Recall 의 부족이 전체 성능을 제한하는 요인으로 작용하였다. 이러한 결과는 Stacking 모델이 기존 단일 모델(Random Forest, XGBoost, LightGBM) 대비 Recall 과 F1-score 측면에서 일부 개선을 보였으나, 여전히 악성 탐지 관점에서는 탐지율 향상이 추가적으로 필요함을 시사한다.

3.4.1.5. 앙상블 결과 분석

세 개의 파인튜닝된 모델을 2 가지의 방법으로 앙상블을 시도했으나, 개별 모델 대비 유의미한 성능 향상은 확인되지 않았다. 일반적으로 앙상블 기법이 효과를 발휘하기 위해서는 각 모델이 서로 다른 오류 패턴을 가져야 한다. 즉, 동일한 데이터에 대해 서로 다른 예측을 수행함으로써 개별 모델의 약점을 보완할 수 있어야 한다. 따라서 앙상블에서 성능 개선이 나타나지 않았다는 것은, 동일한 데이터에 대해 모델 간 예측 결과의 차이가 크지 않았기 때문일 것이라는 가설을 세웠다. 이를 검증하기 위해 각 단일 모델의 prediction 결과를 비교하여, 모델 간 예측 차이가 얼마나 존재하는지 분석하였다. 특히 동일 샘플에 대해 서로 다른 예측을 수행한 케이스를 중심으로 모델 간 다양성을 확인해보았다.

각 단일 모델의 예측 결과를 비교했을 때, 전체 테스트 케이스 500 개 중 예측이 서로 달랐던 케이스는 27 개에 불과한 반면, 세 모델이 모두 동일하게 오답을 낸 케이스는 94 개로 훨씬 많은 비중을 차지하였다. 즉 서로 다른 예측을 통해 보완이 가능한 경우보다, 모든 모델이 동일하게 실패하는 경우가 압도적으로 많았다는 점에서 앙상블의 효과가 제한됐다고 볼 수 있다. 이는

모델들이 공통적으로 어려워하는 데이터 패턴이 존재함을 시사하며, 따라서 모델보다는 데이터 자체의 한계 또는 feature 표현의 부족 문제를 의심하게 되었다.

3.4.1.6. 데이터셋 재구성 실험 배경

앙상블 실험 이후 성능이 기대에 미치지 못한 원인을 파악하기 위해 지도 교수님과 면담을 진행하였다. 다양한 모델과 앙상블 기법을 적용했음에도 성능 향상이 크지 않았다는 점에서, 이는 모델 구조의 문제가 아닌 데이터의 질과 다양성 부족에서 비롯된 한계일 가능성이 높다는 의견을 받았다. 또한 메타데이터 기반 피처 확장이 우선적으로 필요하다는 피드백을 바탕으로, 이후 실험에서는 데이터셋 재구성 및 피처 확장을 중심으로 개선을 진행하였다.
앙상블 모델의 성능 개선 방향을 탐색하는 과정에서 기존 데이터셋을 재검토하였다. 그 결과, 해외여행 관련 정상 URL 데이터 일부에서 경로 구분자인 / 가 . 으로 대체되어 저장된 문제를 확인하였다. 이러한 데이터 이상은 단순 문자열 집계 오류를 넘어, URL 구조 기반 피처인 url_depth, len_first_dir 등의 값이 실제 URL 구조를 반영하지 못하게 만든다는 문제가 있었다. 또한, 기존 kaggle 데이터셋의 경우 URL 수집 시점에 대한 정보가 명확하지 않아, 도메인 생성일, 만료일 등 메타데이터 기반 피처를 활용할 때 데이터 신뢰성 문제가 발생할 가능성이 있다고 판단하였다. 이에 따라 보다 일관된 구조와 수집 정보를 확보할 수 있는 다른 소스의 정상 URL 데이터셋으로 교체하여 실험을 진행하였다.

3.4.1.7. 데이터셋 재구성 과정

정상 URL 데이터셋은 실제 웹 환경과 유사한 정상 URL 데이터를 확보하기 위해 Common Crawl 데이터를 활용하였다. Common Crawl 은 인터넷 전반을 대규모로 크롤링하여 구축된 공개 웹 아카이브로, 특정 서비스나 도메인에 편향되지 않은 다양한 도메인과 URL 구조를 포함하고 있다. 따라서 모델이 실제 환경에서 등장할 수 있는 정상 URL 의 구조적 특성을 학습하는 데 적합하다고 판단했다. 또한 데이터 규모의 단순 증가가 모델의 성능에 유의미한 영향을 끼치지 않을 것이라는 지도 교수님의 의견을 반영하여 데이터의 양은 앞선 실험과 동일한 수준으로 유지하였다. 이에 따라 악성 URL 데이터셋은 기존 데이터를 그대로 사용하고, 정상 URL 데이터셋만 Common Crawl 기반 데이터로 교체하여 실험을 진행하였다.

(1) Baseline 모델 학습
- 정상 URL 데이터 8,000 개
- 악성 URL 데이터 8,000 개

(2) Fine-tuned 모델 학습
- 정상 URL 데이터 400 개
- 악성 URL 데이터 400 개

(3) 테스트 데이터셋
- 정상 URL 데이터 250 개
- 악성 URL 데이터 250 개

3.4.1.8. Stacking 앙상블 결과

앞선 실험에서 Voting 기반 앙상블 모델과 Stacking 기반 앙상블 모델의 성능을 비교한 결과, Voting 모델의 Accuracy 는 0.7173, Stacking 모델의 Accuracy 는 0.7860 으로 나타났다. 이를 통해 Stacking 방식이 개별 모델의 예측 결과를 보다 효과적으로 결합하여 높은 분류 성능을 제공함을 확인할 수 있었다. 따라서 본 연구에서는 보다 우수한 성능을 보인 Stacking 기반 앙상블 방식을 최종 모델 구조로 선정하였으며, 이를 기반으로 추가 실험 및 성능 평가를 진행하였다.

```
PS C:\Users\82102\Desktop\phishing_feature\src> python stacking.py
F1 Score   : 0.9476

Classification Report:
              precision    recall  f1-score   support

           0       0.94      0.96      0.95       250
           1       0.96      0.94      0.95       250

    accuracy                           0.95       500
   macro avg       0.95      0.95      0.95       500
weighted avg       0.95      0.95      0.95       500

Confusion Matrix:
[[239  11]
 [ 15 235]]


================================================
           Model Performance Summary
================================================
Model               ACC        PRE        REC         F1
------------------------------------------------------------
Randomforest     0.9340     0.9657     0.9000     0.9317
XGBoost          0.8860     0.9531     0.8120     0.8769
lightGBM         0.8920     0.9623     0.8160     0.8831
Stacking         0.9480     0.9553     0.9400     0.9476
```

데이터셋 교체 이후 기존 단일 모델들의 성능이 이전보다 전반적으로 향상된 것을 확인할 수 있었다. 특히 정상 여행 URL 데이터의 전처리 오류를 수정한 이후, 모델이 정상 URL 과 악성 URL 의 구조적 차이를 보다 안정적으로 학습하는 경향을 보였다. 이는 URL 구조 기반 피처들이 실제 URL 형태를 보다 정확하게 반영하게 되면서 분류 성능 향상에 긍정적인 영향을 미친 것으로 해석된다.

Stacking 기반 앙상블 모델의 테스트 결과, Accuracy 는 0.9580 으로 나타났다. Precision 은 0.9553 으로 비교적 높은 수준을 유지하여, 악성 URL 로 분류된 경우 실제 악성일 가능성이 높음을 확인할 수 있었다. 이는 기존 실험에서 Accuracy 대비 Precision 이 상대적으로 낮게 나타났던 결과와 달리, 두 지표가 유사한 수준으로 수렴한 결과이며, 정상 URL 데이터의 품질 개선 효과가 반영된 것으로 판단된다.

Recall 은 0.9400 으로 나타나 악성 URL 을 효과적으로 탐지하고 있음을 확인할 수 있었으며, F1-score 는 0.9467 수준으로 측정되었다. 이를 통해 Precision 과 Recall 사이의 균형이 전반적으로

안정적으로 유지되고 있음을 확인할 수 있었다.
또한 Stacking 모델은 기존 단일 모델(Random Forest, XGBoost, LightGBM)과 비교했을 때 Accuracy, Recall, F1-score 측면에서 전반적으로 향상된 성능을 보였다. 이는 서로 다른 모델의 예측 결과를 결합하는 앙상블 구조가 성능 향상에 효과적으로 작용했음을 보여준다.

3.4.2. 프론트엔드

(A) QR 코드 인식 및 URL 추출

```
<CameraView
  style={{ flex: 1 }}
  facing="back"
  barcodeScannerSettings={{
    barcodeTypes: ["qr"],
  }}
  onBarcodeScanned={({ data }) => {
    if (scanned) return;
    if (!data) return;

    setScanned(true);
    router.push(`/ScanResultScreen?url=${encodeURIComponent(data)}`);
  }}
/>
```

QR 코드 인식 이벤트 처리를 위해 onBarcodeScanned 콜백 함수를 사용하였다. 사용자가 QR 코드를 스캔하면 해당 콜백 함수가 실행되며, 스캔된 URL 데이터를 서버 URL 분석 API 요청 모듈로 전달하여 이후 악성 URL 검사 및 결과 출력 과정이 수행되도록 구현하였다.
또한 동일한 QR 코드가 반복적으로 인식되는 문제를 방지하기 위해 React 상태 관리 기능인 const [scanned, setScanned] = useState(false);를 사용하였다. QR 코드가 최초로 스캔되면 scanned 값을 true 로 변경하여 추가 스캔 이벤트를 차단함으로써, 한 번의 스캔만 수행되도록 구현하였다.

(B) 서버 URL 분석 API 요청

```
useEffect(() => {
  const fetchResult = async () => {
    try {
      // URL 검증
      const validation = validateUrl(url);

      if (!validation.isValid || !validation.normalizedUrl) {
        setErrorMessage("유효하지 않은 URL입니다.");
        setError(true);
        return;
      }

      const normalizedUrl = validation.normalizedUrl;
      console.log("검증된 URL:", normalizedUrl);

      // SecureStore에서 JWT 꺼내기
      const token = await SecureStore.getItemAsync("token");
      console.log("토큰 확인:", token);

      if (!token) {
        throw new Error("토큰이 없습니다");
      }

      // 서버에 URL 분석 요청
      const res = await analyzeUrl(normalizedUrl, token);
      setResult(res);
      console.log("서버 응답:", res);
    } catch (e) {
      console.error("URL 분석 실패:", e);
      setError(true);
    }
  };
```

초기 화면 렌더링 시 URL 검증 및 서버 통신이 자동으로 수행되도록 useEffect() 훅을 적용하였다. 컴포넌트가 호출되면 fetchResult() 비동기 함수를 실행하여 URL 유효성 검사와 서버 분석 요청을 순차적으로 진행하도록 구현하였다.

먼저 스캔된 URL 문자열을 URL 유효성 검증 모듈로 전달하여 URL 유효성 검증을 수행하고 검증 결과 URL 형식이 올바르지 않은 경우에는 서버 요청을 보내지 않도록 구성하였다. 유효한 URL 인 경우에는 정규화된 URL 데이터를 사용하여 서버 분석 요청을 진행하도록 구현하였다. 사용자 인증을 위해 expo-secure-store 모듈을 사용하였으며, SecureStore.getItemAsync("token") 함수를 통해 기존 로그인 과정에서 저장된 JWT 토큰을 불러오도록 구성하였다.

URL 유효성은 validateUrl() 함수를 사용하여 검증하였다. 입력값이 비어 있거나 형식이 올바르지 않은 경우에는 서버 요청을 수행하지 않도록 처리하였다. 또한 trim() 및 정규식을 활용하여 공백을 제거하고, 프로토콜이 없는 경우에는 https://를 자동으로 추가하여 URL

형식을 보정하도록 구현하였다. 이후 JavaScript 의 URL 객체를 사용하여 URL 파싱을 수행하고, http 및 https 프로토콜만 허용하도록 제한하였다. 도메인 형식 검사를 위해 hostname.includes(".") 조건을 적용하였으며, 오류 발생 시 예외 메시지를 반환하도록 구성하였다.

(C) URL 분석 결과 시각화

URL 분석 결과 화면은 정상 및 위험 단계를 시각적으로 구분하여 사용자에게 제공하도록 구현하였다. 또한 XAI 분석 결과를 함께 출력하여 사용자가 악성 판단 근거를 이해할 수 있도록 구성하였다. 서버로부터 전달받은 분석 결과는 result 객체에 저장한 뒤 URL 의 악성 여부, 위험 메시지, 특징 정보 등을 사용자 친화적인 형태로 시각화하여 출력하도록 구성하였다. 리다이렉션 정보는 RedirectLog 모듈을 통해 시각화하였다. 사용자가 드롭다운 버튼을 클릭하면 서버에서 전달받은 redirect 데이터를 기반으로 최종 이동 URL, 리다이렉션 횟수, 이동 경로 및 HTTP 상태 코드를 출력하도록 구현하였다. redirect.chain.map()을 사용하여 각 리다이렉션 단계를 반복 렌더링하였으며, 단계별 이동 순서를 원형 번호 UI 로 표시하여 URL 이동 과정을 직관적으로 확인할 수 있도록 구성하였다. 상태 코드는 redirect.status_codes[index] 값을 사용하여 출력하고, 값이 없는 경우 기본값 200 을 사용하도록 처리하였다. 또한 TouchableOpacity 와 Copy 아이콘을 활용하여 URL 복사 버튼을 구현하였다.

```
{/* 안전할 때만 */}
{status === "safe" && (
 <TouchableOpacity
   style={styles.primaryBtn}
    onPress={() => {router.push(`/WebViewScreen?url=${encodeURIComponent(url)}`);}}
 >
 <Text style={{ color: "■#fff"}}>보안 브라우저로 열기</Text>
 </TouchableOpacity>
)}
```

처음 URL 이 안전한 것으로 판별된 경우에는 "보안 브라우저로 열기" 기능을 추가로 제공하였다. router.push()를 사용하여 WebViewScreen 화면으로 이동하도록 구현하였으며, encodeURIComponent(url)을 사용하여 URL 데이터를 안전하게 전달하도록 구성하였다.

(D) 보안 브라우저 JavaScript 제어 구현

보안 브라우저는 react-native-webview 의 javaScriptEnabled 속성을 활용하여 JavaScript 실행 여부를 제어한다. 기본값을 false 로 설정하여 페이지 로드 시 동적 스크립트 실행을 차단하며, 사용자가 JavaScript 활성화 버튼을 누르면 상태값을 true 로 전환한다.

```
<WebView
  key={jsEnabled ? "js-on" : "js-off"}
  ref={webviewRef}
  source={{ uri: url }}
  javaScriptEnabled={jsEnabled}
  onNavigationStateChange={(navState) => {
    setCanGoBack(navState.canGoBack);
  }}
/>
```

단순히 javaScriptEnabled 속성만 변경하면 이미 로드된 WebView 에 즉시 반영되지 않는 문제가 있다. 이를 해결하기 위해 WebView 컴포넌트의 key 속성에 JavaScript 활성화 여부를 바인딩하여, 상태 변경 시 컴포넌트를 완전히 재마운트함으로써 변경된 보안 설정이 새 페이지 로드에 즉시 적용되도록 하였다.

(E) 바텀시트 구현

바텀 시트 내부는 번역 탭과 위험 문구 탭으로 구성되며, 탭 선택 시 접혀있는 상태라면 자동으로 펼쳐지도록 구성하였으며 React Native 의 Animated 와 PanResponder 를 활용하여 구현하였다. @gorhom/bottom-sheet 등의 외부 라이브러리는 네이티브 빌드가 필요하지만, 본 프로젝트의 개발 환경(Expo Go)에서는 네이티브 빌드를 지원하지 않는다. 이에 추가 의존성 없이 순수 React Native API 만으로 동일한 기능을 구현하였다.

```
const snapTo = (expanded: boolean) => {
  Animated.spring(translateY, {
    toValue: expanded ? 0 : EXPANDED_HEIGHT - COLLAPSED_HEIGHT,
    useNativeDriver: true,
    bounciness: 0,
  }).start();
  setIsExpanded(expanded);
};
```

접힌 상태(80px)와 펼쳐진 상태(460px)의 두 가지 스냅 포인트를 정의하고, PanResponder 의 onPanResponderRelease 에서 드래그 이동 거리(dy)가 임계값(50px)을 초과하는 방향에 따라 스냅 위치를 결정한다. 스냅 이동은 Animated.spring 으로 처리하였다.

3.4.3. 백엔드

(A) AI 인터페이스 구현

본 시스템은 머신러닝 모델의 판정 결과와 SHAP 기반 XAI 데이터를 GPT-4o-mini API 를 통해 비전공자 사용자가 이해할 수 있는 자연어 설명으로 변환한다. 단순한 API 호출이 아닌, 도메인 특화 프롬포트 설계를 통해 출력 품질과 일관성을 확보하였다.

1) SHAP 값의 자연어 매핑

모델이 출력하는 feature 이름과 수치를 사전 정의된 매핑 테이블을 통해 사용자가 쉽게 이해할 수 있는 표현으로 변환한다. 또한, SHAP 수치는 방향성(위험/안전 신호)만 추출하고 원시 수치는 노출하지 않는다.

```
FEATURE_LABELS = {
    "url_length":      "웹 주소가 비정상적으로 길어요",
    "special_chars":   "주소에 이상한 기호가 많아요",
    "domain_age":      "최근에 만들어진 사이트예요",
    "https_used":      "보안 연결이 되어 있어요",
    "redirect_count": "클릭하면 다른 곳으로 자동 이동해요",
    "known_domain":    "알려진 사이트 주소가 아니에요",
}
```

2) Few-shot 프롬포팅 적용

출력 일관성 확보를 위해 Few-shot 기법을 적용하였다. 규칙만으로는 GPT 가 말투와 길이를 일관되게 유지하기 어렵기 때문에, 타겟 사용자(20~40 대 여행자) 기준의 출력 예시를 프롬포트 내에 포함시켜 원하는 톤과 구조를 학습시켰다.

이를 통해 전문용어 배제, 여행 맥락 반영, 3 단 구조(근거-판단-행동) 출력을 안정적으로 유도한다.

```
FEW_SHOT_EXAMPLES = """
[예시 1]
판정: 피싱 의심
의심 이유: 웹 주소가 비정상적으로 길어요 / 주소에 이상한 기호가 많아요
출력:
1. 이 링크는 항공사 예약 사이트처럼 보이지만, 주소가 수상할 정도로 길고 알아보기 힘든 기호들이 섞여 있어요.
2. 개인정보나 결제 정보를 훔치려는 가짜 사이트일 가능성이 높아요.
3. 클릭하지 말고 항공사 공식 앱이나 즐겨찾기에 저장된 주소로 직접 들어가세요.

[예시 2]
판정: 정상
의심 이유: 없음
출력:
1. 이 링크는 잘 알려진 호텔 예약 사이트로, 주소 형태도 정상적이고 안전하게 연결돼요.
2. 현재까지 위험한 신호가 발견되지 않은 안전한 링크예요.
3. 접속해도 괜찮지만, 결제 전에 주소창에 자물쇠 아이콘이 있는지 한 번만 확인해보세요.
```

3) 구조화된 출력 형식 강제(CoT 기반)

GPT 의 자유 생성을 허용하면 출력 구조가 매번 달라지는 문제가 있다. 이를 해결하기 위해 Chain-of-Thought 방식으로 출력 섹션을 명시적으로 강제한다.

본 모듈의 핵심은 XAI 출력을 단순 번역하는 것이 아니라 사용자 맥락에 맞게 재구성하는 데 있다. SHAP 수치를 그대로 노출하는 대신 위험 신호의 의미를

해외여행 상황에 맞는 언어로 재표현함으로써, 비전공자도 모델의 판단 근거를 직관적으로 이해할 수 있도록 설계하였다.

(B) 리다이렉션 추적 구현

URL 분석 모듈은 Python 의 requests 라이브러리를 활용하여 입력 URL 의 실제 이동 경로를 추적한다. allow_redirects=True 옵션을 통해 HTTP 요청 시 발생하는 모든 리다이렉션을 자동으로 따라가도록 설정하였으며, resp.history 에 기록된 중간 경유 응답 목록과 최종 도달 URL 을 수집하여 전체 이동 경로를 구성한다.
수집되는 데이터는 다음과 같다.

- final_url: 최종 도달한 URL
- redirect_count: 리다이렉션 발생 횟수
- chain: 경유 URL 전체 목록 (시작 URL 포함)
- status_codes: 각 경유 지점의 HTTP 상태 코드

피싱 사이트는 정상 URL 로 위장한 뒤 다단계 리다이렉션을 통해 악성 페이지로 유도하는 방식을 자주 사용한다. 입력 URL 만으로는 최종 목적지를 알 수 없기 때문에, 실제 이동 경로를 추적하여 AI 모델 및 GPT 설명 생성 단계에 함께 전달하도록 설계하였다.
봇 차단으로 인한 분석 실패를 방지하기 위해 User-Agent 헤더를 일반 브라우저로 설정하였으며, 네트워크 불안정 환경을 고려하여 타임아웃을 5 초로 제한하였다. 또한, Timeout, SSLError 등 접속 실패 상황에 대한 예외 처리를 구성하여, 오류 발생 시에도 분석 파이프라인이 중단되지 않고 에러 원인과 함께 원본 URL 을 반환하도록 하였다.